机器人ROS基础与实践

第三章：ROS2 话题通信编程



目录

1. ROS2工作空间与功能包

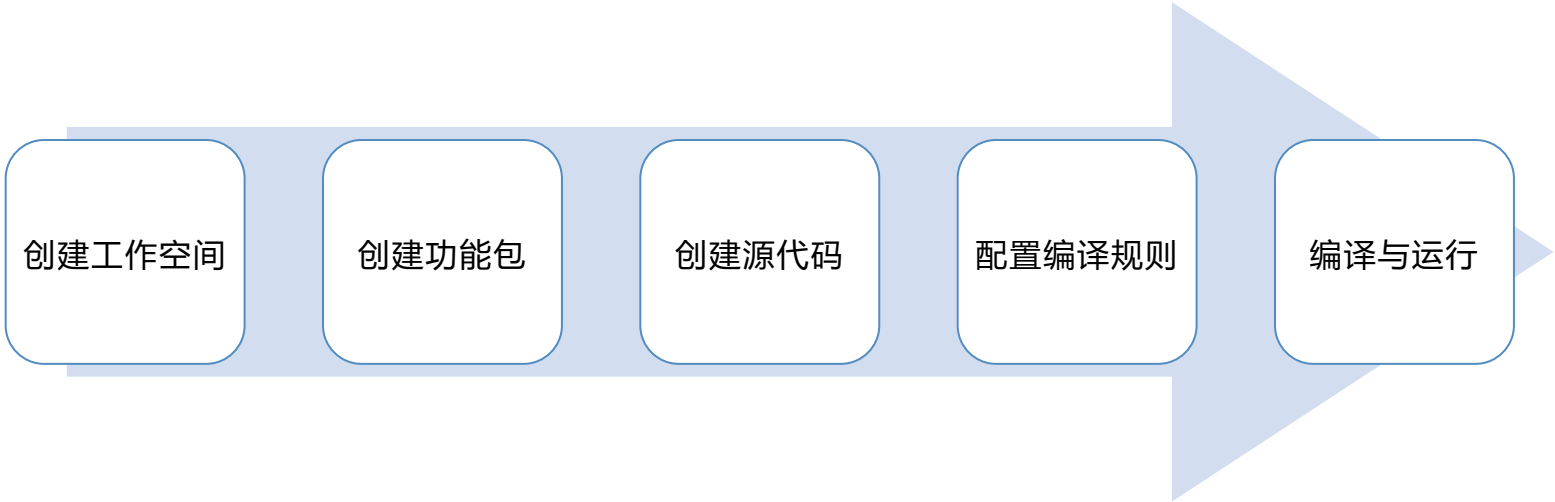
2. 话题通信编程：Publisher / Subscriber

3. 本章小结



1. ROS2工作空间与功能包

1.1 ROS项目开发流程



1.2 什么是工作空间（Workspace）？

- 工作空间是ROS2项目的根目录，包含所有源代码、构建输出和安装文件。

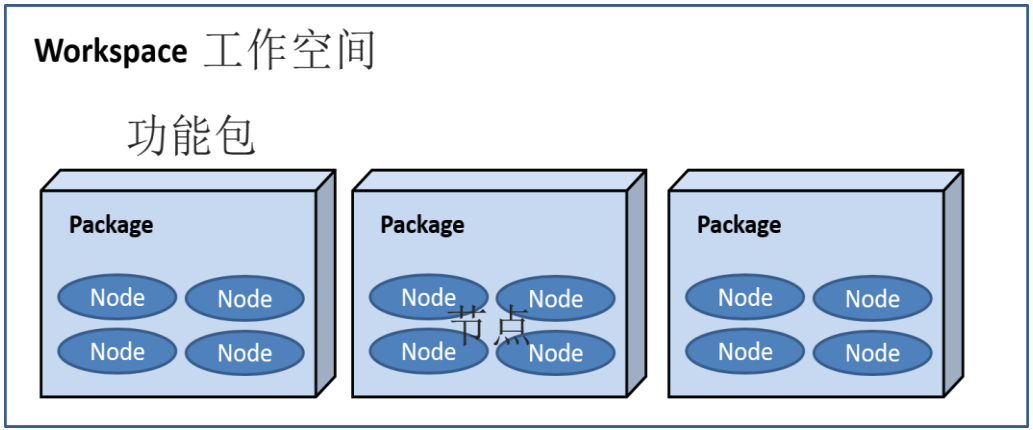
典型的工作空间目录结构：

```
ros2_ws/                          ← 工作空间根目录
├── src/                          ← 源代码目录
│   ├── my_robot_bringup/         ← 包1
│   ├── my_robot_driver/          ← 包2
│   └── my_robot_control/         ← 包3
├── build/                        ← CMake构建目录（自动生成）
├── install/                      ← 安装目录（自动生成）
└── log/                          ← 日志目录（自动生成）
```

https://blog.csdn.net/weixin_52694742/article/details/156242960

1.3 什么是功能包（Package）？

- 功能包是工作空间的一个组件，也就是ROS2的基本组织单元；
- 一个工作空间可以包含多个功能包；一个功能包下面可以有多个节点。




```
my_robot_ws/                    # 工作空间名称可自定义
|
├── src/                        # 源代码目录
|   ├── package1/
|   |   ├── src/                # C++源文件目录
|   |   ├── include/            # C++头文件目录
|   |   ├── launch/             # 启动文件
|   |   ├── config/             # 配置文件
|   |   ├── CMakeLists.txt      # CMake构建配置（C++）
|   |   └── package.xml         # 包元数据
|   |
|   └── package2/
|       ├── package2/           # Python包目录（Python专用）
|       |   ├── __init__.py
|       |   └── nodes/
|       ├── resource/           # 资源文件
|       ├── test/               # 测试文件
|       ├── setup.py            # Python安装配置
|       ├── setup.cfg           # 包配置
|       ├── package.xml         # 包元数据
|       └── launch/             # 启动文件
|
├── build/                      # 构建输出目录（colcon生成）
|   ├── package1/
|   └── package2/
|
├── install/                    # 安装目录（colcon生成）
|   ├── package1/
|   ├── package2/
|   └── local_setup.bash        # 工作空间环境设置
|
└── log/                        # 构建日志（colcon生成）
    └── latest_build/
```

https://blog.csdn.net/weixin_52694742/article/details/156242960

https://zhangrelay.blog.csdn.net/article/details/119075451

1.4 工作空间的创建和编译

创建工作空间

```
$ mkdir -p ~/ros2_ws/src
```

编译工作空间

```
$ cd ~/ros2_ws && colcon build
```

设置环境变量

```
$ source install/setup.bash
```

检查环境变量

```
$ echo $AMENT_PREFIX_PATH
```

查看目录树

```
$ tree -L 2
```

```
.
├── build
│   └── COLCON_IGNORE
├── install
│   ├── COLCON_IGNORE
│   ├── local_setup.bash
│   ├── local_setup.ps1
│   ├── local_setup.sh
│   ├── _local_setup_util_ps1.py
│   ├── _local_setup_util_sh.py
│   ├── local_setup.zsh
│   ├── setup.bash
│   ├── setup.ps1
│   ├── setup.sh
│   └── setup.zsh
├── log
│   ├── build_2026-06-17_09-13-23
│   ├── COLCON_IGNORE
│   ├── latest -> latest_build
│   └── latest_build -> build_2026-06-17_09-13-23
└── src
```

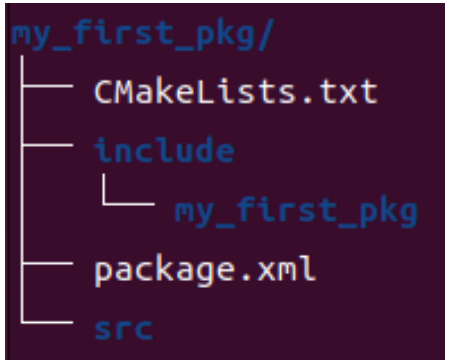
1.5 创建第一个包

```
$ cd ~/ros2_ws/src

# 创建C++包
$ ros2 pkg create --build-type ament_cmake my_first_pkg

# 或创建Python包
$ ros2 pkg create --build-type ament_python my_py_pkg

# 查看目录树
$ tree my_first_pkg/
```

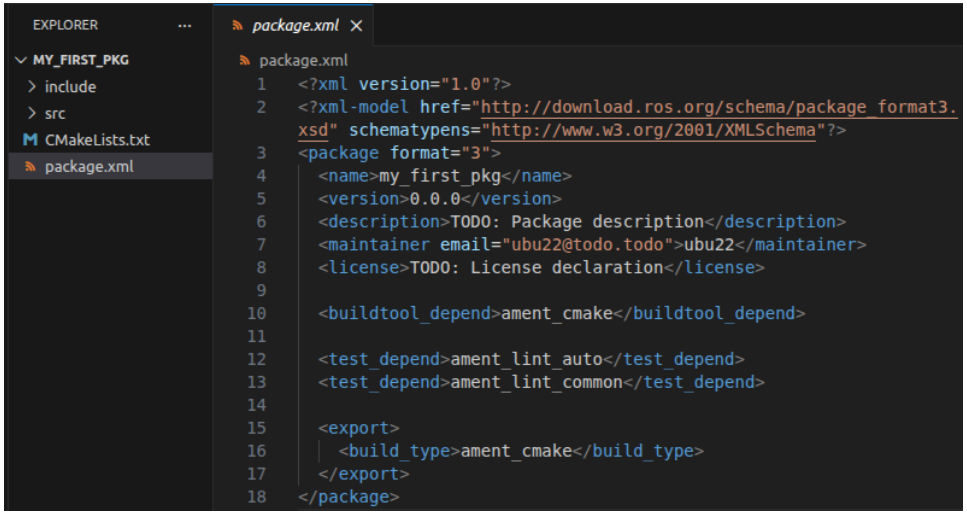


- 注意：
  - 同一个工作空间下，不允许存在同名功能包
  - 不同工作空间下，允许存在同名功能包

1.5 创建第一个包

- C++包的package.xml示例

空白功能包描述



相对完整的功能包描述

```xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" sch
<package format="3">
  <!-- 包的基本信息 -->
  <name>my_first_pkg</name>
  <version>0.0.1</version>
  <description>我的第一个ROS2包</description>

  <!-- 作者和维护者信息 -->
  <maintainer email="your_email@example.com">Your Name</maintainer>
  <author email="your_email@example.com">Your Name</author>

  <!-- 开源协议 -->
  <license>Apache-2.0</license>

  <!-- 构建系统 -->
  <buildtool_depend>ament_cmake</buildtool_depend>

  <!-- 编译时依赖 -->
  <build_depend>rclcpp</build_depend>
  <build_depend>std_msgs</build_depend>

  <!-- 运行时依赖 -->
  <exec_depend>rclcpp</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <!-- 测试依赖 -->
  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_cmake_gtest</test_depend>

  <!-- 导出依赖（为其他包提供库） -->
  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

1.5 创建第一个包

```
# 编译
$ cd ~/ros2_ws && colcon build

# 设置环境变量
$ source install/setup.bash

# 检查环境变量
$ echo $AMENT_PREFIX_PATH
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~/ros2_ws$ echo $AMENT_PREFIX_PATH
/home/ubu22/ros2_ws/install/my_first_pkg:/opt/ros/humble
```

1.5 创建第一个包

```
# 编译
$ cd ~/ros2_ws && colcon build

# 设置环境变量
$ source install/setup.bash

# 检查环境变量
$ echo $AMENT_PREFIX_PATH

# 查看当前环境可用包
$ ros2 pkg list
$ ros2 pkg list |grep my_first
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~/ros2_ws$ ros2 pkg list |grep my_first
my_first_pkg
```

1.5 创建第一个包

```
# 编译
$ cd ~/ros2_ws && colcon build

# 设置环境变量
$ source install/setup.bash

# 检查环境变量
$ echo $AMENT_PREFIX_PATH

# 查看当前环境可用包
$ ros2 pkg list
$ ros2 pkg list |grep my_first

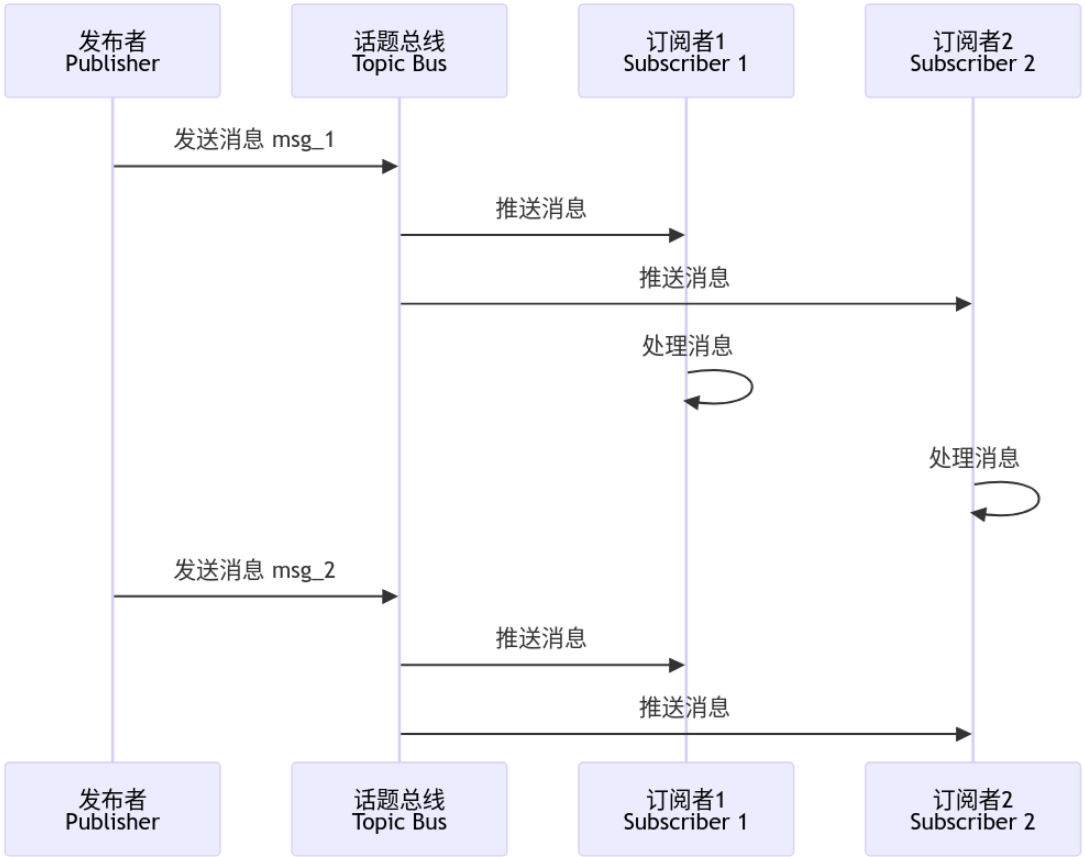
# 查看目录树
$ tree -L 2
```

```
├── build
│   ├── COLCON_IGNORE
│   └── my_first_pkg
├── install
│   ├── COLCON_IGNORE
│   ├── local_setup.bash
│   ├── local_setup.ps1
│   ├── local_setup.sh
│   ├── _local_setup_util_ps1.py
│   ├── _local_setup_util_sh.py
│   ├── local_setup.zsh
│   ├── my_first_pkg
│   ├── setup.bash
│   ├── setup.ps1
│   ├── setup.sh
│   └── setup.zsh
├── log
│   ├── build_2026-06-17_09-13-23
│   ├── build_2026-06-17_09-29-54
│   ├── COLCON_IGNORE
│   ├── latest -> latest_build
│   └── latest_build -> build_2026-06-17_09-29-54
└── src
    └── my_first_pkg
```



2. 话题通信编程

2.1 话题通信时序示意图



2.2 话题通信程序开发流程

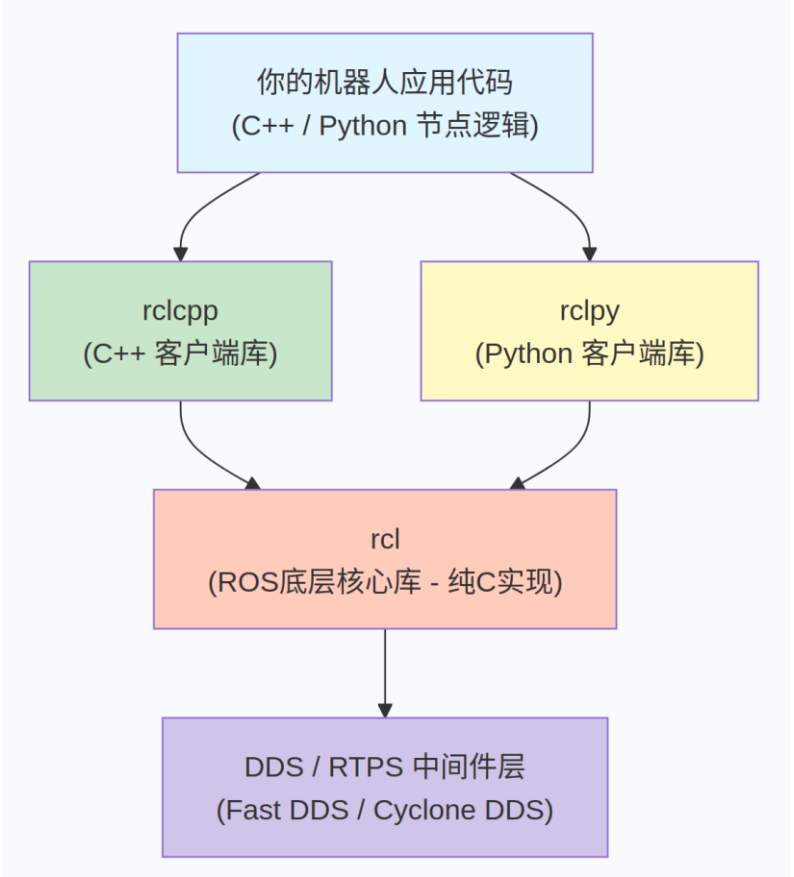
- 任务
  - 编写一个ROS2话题通信的程序，创建一个发布者（publisher）发送"counter"计数消息，和一个订阅者（subscriber）接收并打印这条消息
- 主要步骤
  - 创建工作空间和功能包
  - 编写发布者节点代码
  - 编写订阅者节点代码
  - 修改CMakeLists.txt
  - 编译运行

2.3 创建工作空间和功能包

```
# 创建工作空间
$ mkdir -p ~/ros2_ws/src

# 创建功能包（依赖 rclcpp 和 std_msgs）
$ ros2 pkg create pub_sub_demo --build-type
ament_cmake --dependencies rclcpp std_msgs
```

- rclcpp 是 ROS 2 的 C++ 客户端库，为 C++ 开发者提供访问 ROS 2 核心功能（如话题、服务、参数）的 API
- rclpy 是 ROS 2 的 Python 客户端库，为 Python 开发者提供同样的功能，但基于 Python 语言特性封装



2.4 编写发布者和订阅者节点代码

创建工作空间

$ mkdir -p ~/ros2_ws/src

创建功能包（依赖 rclcpp 和 std_msgs）

$ ros2 pkg create pub_sub_demo --build-type ament_cmake --dependencies rclcpp std_msgs

创建和编写发布者、订阅者节点代码

$ cd ~/ros2_ws/src/pub_sub_demo/src/

$ touch publisher.cpp

$ touch subscriber.cpp

src > publisher.cpp

```cpp
#include <chrono>
#include <iostream>
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/int32.hpp"

// 使用命名空间
using namespace std::chrono_literals;

class PublisherNode : public rclcpp::Node {
public:
    PublisherNode() : Node("cpp_talker"), counter_(0) {
        // 创建发布者
        publisher_ = this->create_publisher<std_msgs::msg::Int32>("counter", 10);

        // 创建定时器，每秒执行一次
        timer_ = this->create_wall_timer(
            1000ms,
            std::bind(&PublisherNode::timer_callback, this)
        );

        RCLCPP_INFO(this->get_logger(), "C++ 发布者节点已启动");
    }

private:
    void timer_callback() {
        auto msg = std_msgs::msg::Int32();
        msg.data = counter_++;

        publisher_->publish(msg);
        RCLCPP_INFO(this->get_logger(), "发送消息: %d", msg.data);
    }

    rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    int counter_;
};

int main(int argc, char *argv[]) {
    // 初始化ROS2
    rclcpp::init(argc, argv);

    // 创建并运行节点
    rclcpp::spin(std::make_shared<PublisherNode>());

    // 关闭ROS2
    rclcpp::shutdown();

    return 0;
}
```

2.4 编写发布者和订阅者节点代码

创建工作空间

$ mkdir -p ~/ros2_ws/src

创建功能包（依赖 rclcpp 和 std_msgs）

$ ros2 pkg create pub_sub_demo --build-type ament_cmake --dependencies rclcpp std_msgs

创建和编写发布者、订阅者节点代码

$ cd ~/ros2_ws/src/pub_sub_demo/src/

$ touch publisher.cpp

$ touch subscriber.cpp

https://blog.csdn.net/weixin_52694742/article/details/156242972

src > subscriber.cpp

```cpp
#include <iostream>
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/int32.hpp"

class SubscriberNode : public rclcpp::Node {
public:
    SubscriberNode() : Node("cpp_listener"), message_count_(0) {
        // 创建订阅者
        subscription_ = this->create_subscription<std_msgs::msg::Int32>(
            "counter",
            10,
            std::bind(&SubscriberNode::listener_callback,
                      this, std::placeholders::_1)
        );

        RCLCPP_INFO(this->get_logger(), "C++ 订阅者节点已启动");
    }

private:
    void listener_callback(const std_msgs::msg::Int32 & msg) {
        message_count_++;
        RCLCPP_INFO(this->get_logger(), "收到消息 #%d: %d",
                    message_count_, msg.data);
    }

    rclcpp::Subscription<std_msgs::msg::Int32>::SharedPtr subscription_;
    int message_count_;
};

int main(int argc, char *argv[]) {
    // 初始化ROS2
    rclcpp::init(argc, argv);

    // 创建并运行节点，让节点进入循环，处理回调
    rclcpp::spin(std::make_shared<SubscriberNode>());

    // 关闭ROS2
    rclcpp::shutdown();
    return 0;
}
```

2.5 修改CMakeLists.txt

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(pub_sub_demo)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

# 发布者可执行文件
add_executable(publisher src/publisher.cpp)
ament_target_dependencies(publisher rclcpp std_msgs)

# 订阅者可执行文件
add_executable(subscriber src/subscriber.cpp)
ament_target_dependencies(subscriber rclcpp std_msgs)

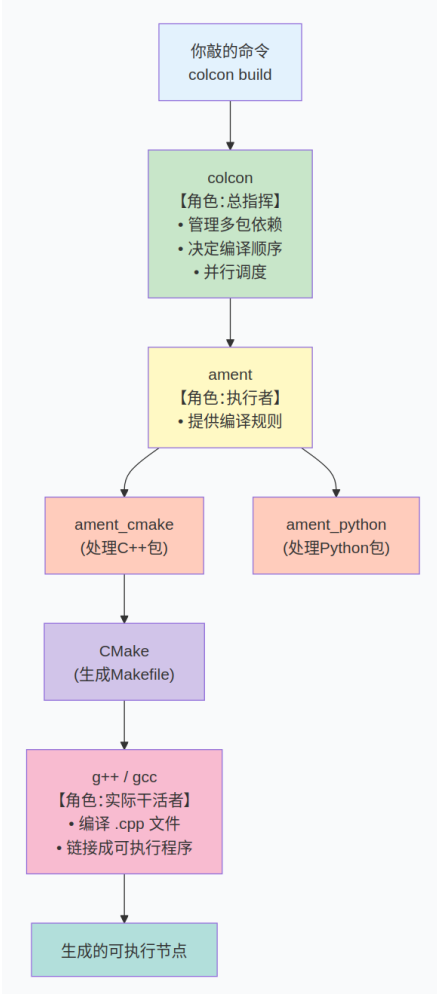
# 安装可执行文件到 lib/${PROJECT_NAME}/ 目录
install(TARGETS
  publisher
  subscriber
  DESTINATION lib/${PROJECT_NAME}
)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

自动生成

手动添加





2.6 编译和运行

```
# 编译
$ cd ~/ros2_ws
$ colcon build --packages-select pub_sub_demo
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~/ros2_ws$ colcon build --packages-select pub_sub_demo
[0.134s] WARNING:colcon.colcon_ros.prefix_path.ament:The path '/home/ubu22/ros2_ws/ins
[0.134s] WARNING:colcon.colcon_ros.prefix_path.catkin:The path '/home/ubu22/ros2_ws/in
Starting >>> pub_sub_demo
Finished <<< pub_sub_demo [4.47s]

Summary: 1 package finished [4.59s]
```

2.6 编译和运行

```
# 编译
$ cd ~/ros2_ws
$ colcon build --packages-select
pub_sub_demo

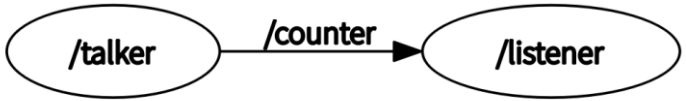
# 设置环境变量
$ source install/setup.bash

# 运行发布者节点
$ ros2 run pub_sub_demo publisher

# 运行订阅者节点（另开一个终端）
$ cd ~/ros2_ws && source install/setup.bash
$ ros2 run pub_sub_demo subscriber

# 查看计算图
$ rqt_graph
```

```
ubu22@ubu22-Legion-Y9000P-IRX9:~/ros2_ws$ ros2 run pub_sub_demo publisher
[INFO] [1781666195.818523423] [cpp_talker]: C++ 发布者节点已启动
[INFO] [1781666196.818836533] [cpp_talker]: 发送消息: 0
[INFO] [1781666197.818850206] [cpp_talker]: 发送消息: 1
[INFO] [1781666198.818890231] [cpp_talker]: 发送消息: 2
[INFO] [1781666199.818906321] [cpp_talker]: 发送消息: 3
[INFO] [1781666200.818861296] [cpp_talker]: 发送消息: 4
[INFO] [1781666201.818948129] [cpp_talker]: 发送消息: 5
[INFO] [1781666202.818796773] [cpp_talker]: 发送消息: 6
```




```
ubu22@ubu22-Legion-Y9000P-IRX9:~/ros2_ws$ ros2 run pub_sub_demo subscriber
[INFO] [1781666290.484457042] [cpp_listener]: C++ 订阅者节点已启动
[INFO] [1781666290.561880524] [cpp_listener]: 收到消息 #1: 3
[INFO] [1781666291.561796938] [cpp_listener]: 收到消息 #2: 4
[INFO] [1781666292.561933608] [cpp_listener]: 收到消息 #3: 5
[INFO] [1781666293.561858578] [cpp_listener]: 收到消息 #4: 6
[INFO] [1781666294.561807621] [cpp_listener]: 收到消息 #5: 7
[INFO] [1781666295.561804194] [cpp_listener]: 收到消息 #6: 8
```



2.7 创建python版本的发布订阅demo

```
# 创建功能包（依赖 rclpy 和 std_msgs）
$ cd ~/ros2_ws/src
$ ros2 pkg create pub_sub_py --build-type ament_python --dependencies rclpy std_msgs
$ cd pub_sub_py/pub_sub_py

# 在pub_sub_py/pub_sub_py目录下新建 publisher.py、subscriber.py节点文件
```

2.7 创建python版本的发布订阅demo

publisher.py

src > pub_sub_py > pub_sub_py > publisher.py > ...

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import rclpy                                    # ROS2 Python接口库
from rclpy.node import Node                     # ROS2 节点类
from std_msgs.msg import Int32                  # int消息类型

"""
创建一个发布者节点
"""
class PublisherNode(Node):

    def __init__(self, name):
        self.counter = 0                                        # 计数器，保存为实例属性以便回调中累加
        super().__init__(name)                                  # ROS2节点父类初始化
        self.pub = self.create_publisher(Int32, "counter", 10)  # 创建发布者对象（消息类型、话题名、队列长度）
        self.timer = self.create_timer(0.5, self.timer_callback)  # 创建一个定时器（单位为秒的周期，定时执行的回调函数）

    def timer_callback(self):                                   # 创建定时器周期执行的回调函数
        msg = Int32()                                           # 创建一个int类型的消息对象
        self.counter = self.counter + 1
        msg.data = self.counter                                 # 填充消息对象中的消息数据
        self.pub.publish(msg)                                   # 发布话题消息
        self.get_logger().info(f'发送消息: {msg.data}')          # 输出日志信息，提示已经完成话题发布

def main(args=None):                                # ROS2节点主入口main函数
    rclpy.init(args=args)                           # ROS2 Python接口初始化
    node = PublisherNode("topic_py_publisher")      # 创建ROS2节点对象并进行初始化
    rclpy.spin(node)                                # 循环等待ROS2退出
    node.destroy_node()                             # 销毁节点对象
    rclpy.shutdown()                                # 关闭ROS2 Python接口
```

2.7 创建python版本的发布订阅demo

subscriber.py

src > pub_sub_py > pub_sub_py > subscriber.py > ...

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import rclpy                                    # ROS2 Python接口库
from rclpy.node   import Node                   # ROS2 节点类
from std_msgs.msg import Int32                  # ROS2标准定义的int32消息

"""
创建一个订阅者节点
"""
class SubscriberNode(Node):

    def __init__(self, name):
        super().__init__(name)                                  # ROS2节点父类初始化
        self.sub = self.create_subscription(\
            Int32, "counter", self.listener_callback, 10)       # 创建订阅者对象（消息类型、话题名、订阅者回调函数、队列长度）

    def listener_callback(self, msg):                           # 创建回调函数，执行收到话题消息后对数据的处理
        self.get_logger().info('收到消息: "%d"' % msg.data)      # 输出日志信息，提示订阅收到的话题消息

def main(args=None):                                # ROS2节点主入口main函数
    rclpy.init(args=args)                           # ROS2 Python接口初始化
    node = SubscriberNode("topic_py_publisher")     # 创建ROS2节点对象并进行初始化
    rclpy.spin(node)                                # 循环等待ROS2退出
    node.destroy_node()                             # 销毁节点对象
    rclpy.shutdown()                                # 关闭ROS2 Python接口

```

2.7 创建python版本的发布订阅demo

```
# 创建功能包（依赖 rclpy 和 std_msgs）
$ cd ~/ros2_ws/src
$ ros2 pkg create pub_sub_py --build-type ament_python --dependencies rclpy std_msgs
$ cd pub_sub_py/pub_sub_py

# 在pub_sub_py/pub_sub_py目录下新建 publisher.py、subscriber.py节点文件

# 修改src/pub_sub_py/setup.py 的entry_points 注册可执行节点以支持ros2 run调用
```

```
entry_points={
    'console_scripts': [
    'publisher = pub_sub_py.publisher:main',
    'subscriber = pub_sub_py.subscriber:main'
    ],
},
```

2.7 创建python版本的发布订阅demo

```
# 创建功能包（依赖 rclpy 和 std_msgs）
$ cd ~/ros2_ws/src
$ ros2 pkg create pub_sub_py --build-type ament_python --dependencies rclpy
std_msgs
$ cd pub_sub_py/pub_sub_py

# 在pub_sub_py/pub_sub_py目录下新建 publisher.py、subscriber.py节点文件
# 修改src/pub_sub_py/setup.py 的entry_points 注册可执行节点以支持ros2 run调用

# 编译运行
$ cd ~/ros2_ws && colcon build && source  install/setup.bash
$ ros2 run pub_sub_py publisher

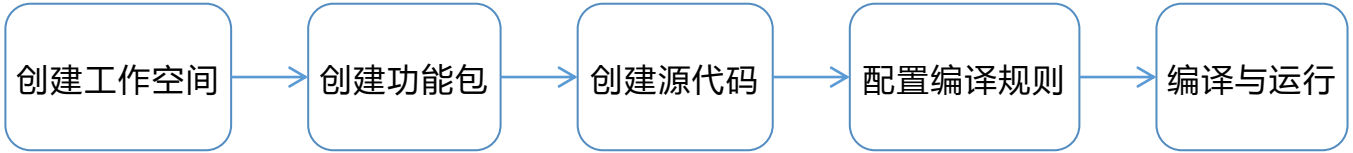
# 另一个窗口运行
$ ros2 run pub_sub_py subscriber
```



3. 本节小结

本节小结

ROS项目开发流程



话题通信编程

创建工作空间和功能包->编写发布者节点代码->编写订阅者节点代码->
修改CMakeLists.txt->编译->设置环境变量->
运行发布者节点->运行订阅者节点

感谢倾听！

Thanks for Listening

